

AI Destination OS — Projektkontext für Claude Code

Aktueller Stand (bitte zuerst lesen)

Dieses Repo ist kein leeres Grundgerüst — es enthält bereits einen **echten, getesteten Teil** der Plattform. Bitte vor jeder neuen Aufgabe kurz prüfen, was schon existiert, statt etwas doppelt zu bauen:

Teil	Status	Details
Backend (Fast API)	✓ Fertig & getestet	78/78 pytest-Tests grün (<code>docker compose exec backend pytest tests/ -v</code> ; Stand: aktualisiert nach realem Docker-Testlauf, vorheriger Stand von 62 war veraltet). Auth, Tenants, POIs (inkl. Delete), Trips, Chat, RAG (lexikalischer Fallback), Content-Pipeline, Analytics, Booking-Agent mit Consent-Flow, CV-Stub, Constraint-Solver (17 Tests) — siehe Prinzip 6.
Frontend (Next.js)	✓ Fertig & E2E-getestet	13/13 Playwright-E2E-Tests grün (<code>docker compose --profile test run --rm playwright</code>) — Landing-Page, Login/Register-Flow, Demo-Login, alle 5 Tabs (Home/Explorer/Réservations/Assistant/Profil), Logout, Fehlerbehandlung bei ungültigem Token. Wichtige Korrektur ggü. früherem Stand: Es sind KEINE 4 separaten Seiten/Routen (Home/POIs/Trips/Chat) — das reale UI ist eine Single-Page-App mit genau einer Next.js-Route (<code>/</code>), Tabs laufen komplett über React-State in <code>AppShell.tsx</code> (kein URL-Routing zwischen Tabs). <code>next.config.js</code> proxied <code>/api/*</code> serverseitig zum Backend (Rewrite, konfigurierbar über <code>BACKEND_URL</code> env var — nicht <code>NEXT_PUBLIC_API_URL</code> , das wird für diesen Pfad nicht genutzt).
Mobile (Flutter)	✗ Nicht begonnen	Kompletter Aufbau nötig: <code>mobile/lib/{models,providers,screens,widgets,services}</code> .
Docker / Docker Compose	✓ Fertig & getestet	<code>docker-compose.yml</code> (Postgres+PostGIS+pgvector, Redis, Backend, Frontend, Playwright unter <code>profiles: ["test"]</code>) steht, <code>docker compose up --build</code> läuft durch, alle Tests grün im Container. Kein MinIO/S3 — im Code bislang nicht verwendet, bei Bedarf später ergänzen.
Kubernetes Manifeste	✗ Nicht begonnen	<code>deploy/k8s/</code> existiert noch nicht.
CI/CD (GitHub Actions)	✗ Nicht begonnen	<code>.github/workflows/ci.yml</code> existiert noch nicht.
LLM / CV - Anbi	● Abstrahiert, nicht	Code ist auf <code>OPENAI_API_KEY</code> / <code>LLM_PROVIDER</code> vorbereitet (siehe <code>app/config.py</code> , <code>app/services/rag_service.py</code> , <code>app/services/chat_service.py</code>), aber es wurde noch kein echter Provider-Call gemacht — läuft aktuell im deterministischen Mock-Modus.

Teil	Status	Details
ndun g	angeb unden	

⚠ **Kritischer Hinweis zum Arbeitsverzeichnis:** Es existierten zeitweise **mehrere divergierende Kopien** dieses Repos auf der Entwickler-Maschine (u. a. unter `~/Downloads/discover-ai-source-repo(1)/(2)/(3)` , `~/Downloads/files` , sowie die eigentliche Arbeitskopie unter `~/Documents/Projet invest/Dicouver App/discover-ai-source-repo(3)`). **Dieses Verzeichnis ist bislang KEIN Git-Repository** (`git init` wurde noch nicht ausgeführt) — alle hier dokumentierten Fixes existieren nur lokal auf der Maschine des Entwicklers, nicht versioniert. Vor jeder neuen Session: zuerst `pwd` und `git status` prüfen, um sicherzustellen, dass an der richtigen, aktuellen Kopie gearbeitet wird. **Dringend empfohlen:** `git init && git add . && git commit` in der Arbeitskopie, sobald möglich, um diesen Stand dauerhaft zu sichern.

Bekannte, in dieser Session behobene Bugs (zur Referenz, falls sie in einer nicht synchronisierten Kopie erneut auftauchen):

- `frontend/src/app/page.tsx` : `useAuth()` wurde mit falschem Property-Namen destrukturiert (`isLoading` statt `loading`) — TypeScript-Build-Fehler.
- `frontend/next.config.js` : `API-Rewrite-Destination` war fest auf `http://localhost:8000` verdrahtet — im Docker-Netzwerk unerreichbar von innerhalb des Frontend-Containers. Jetzt konfigurierbar über `process.env.BACKEND_URL` .
- `docker-compose.yml` : Playwright-Service brauchte einen anonymen Volume-Mount für `/app/node_modules` , um Konflikte mit dem host-seitigen `node_modules` beim Bind-Mount zu vermeiden.

Empfohlene Reihenfolge für die nächsten Schritte:

- ~~Docker Compose (dev) schreiben + Backend- und Frontend-Dockerfile real bauen und testen~~ ✓ erledigt
- ~~Frontend-Rendering im echten Browser verifizieren~~ ✓ erledigt — 13/13 Playwright-Tests grün
- Repo als Git-Repository initialisieren und aktuellen Stand committen** — noch offen, siehe Hinweis oben
- Flutter-App aufbauen (kompletter neuer Block — als eigene Teilaufgabe behandeln, nicht nebenbei)
- CI/CD-Pipeline (GitHub Actions): mindestens `pytest` + `npm run build` + `playwright test` bei jedem PR
- Kubernetes-Manifeste für Produktion
- Echten LLM-/CV-Provider hinter die bestehenden Abstraktionen hängen

Hinweis zur Pflege dieser Tabelle: Der Testcount und der Docker-Status waren zuvor veraltet (62 statt real 78 Tests, "nie gebaut" obwohl inzwischen erfolgreich gebaut). Auch die Frontend-Architektur war falsch beschrieben (4 separate Seiten statt Single-Page-App mit Tab-State). Bitte nach größeren Sessions kurz gegenchecken, ob diese Tabelle noch mit der Realität übereinstimmt — sonst arbeitet die nächste Claude-Code-Session mit falschen Annahmen.

Ausführliche Architektur-Begründung: siehe `docs/architektur.md` .

Was dieses Projekt ist

Ein KI-gestütztes Destination Operating System für den Tourismussektor: persönlicher Reiseberater (Planung, CV/AR-Vor-Ort-Guide, Live-Kontext, Buchungsagent) plus B2G-Dashboard für Hotels/Behörden/Tourismusbüros. Von Beginn an als **internationale Multi-Tenant-/White-Label-Plattform** konzipiert, nicht als Einzelmarkt-App.

Vollständige Architektur: siehe `docs/architektur.md` (aus dem Strategiedokument übernommen).

Nicht verhandelbare Architekturprinzipien

Diese Regeln gelten für **jede** Änderung, unabhängig davon, wie klein die Aufgabe wirkt:

- Kein länder- oder kundenspezifischer Code im Kern.** Alles, was sich zwischen Märkten/Tenants unterscheidet (Branding, Sprache, Zahlungsanbieter, Feature-Umfang, Content), ist **Konfiguration oder Daten**, niemals ein `if country == "DZ"` im Kerncode. Wenn eine Aufgabe das nahelegt, zuerst zurückfragen statt zu implementieren.
- tenant_id auf allen Kernentitäten.** User, Bookings, Sessions, Konfigurationsobjekte — jede neue Tabelle/jedes neue Model bekommt von Anfang an ein Tenant-Feld, auch wenn aktuell nur ein Tenant existiert.

- 3. **RAG–Wissensbasis strikt pro Tenant getrennt.** Kein gemeinsamer Vektorindex über mehrere Länder/Kunden hinweg — sonst sickert Kontext aus Markt A in Anfragen aus Markt B.
- 4. **Externe Anbieter nur über einen Adapter–Layer.** Karten, Buchungssysteme, Zahlungsanbieter, Wetter–/Verkehrsdaten — jede Integration bekommt ein Interface, hinter dem der konkrete Anbieter austauschbar ist. Kein direkter Aufruf einer Drittanbieter–API aus der Kernlogik.
- 5. **Booking–Agent strikt getrennt von Empfehlungslogik.** Jede Aktion mit finanzieller/vertraglicher Wirkung (Buchung, Kauf, Stornierung) braucht einen expliziten Consent–Schritt im Flow — niemals eine stille Automatisierung, auch wenn das technisch möglich wäre.
- 6. **Harte Constraints (Öffnungszeiten, Budget, Distanzen) laufen über einen Solver/Regel–Layer, nicht über das LLM.** Das LLM schlägt vor und formuliert; die Einhaltung harter Zahlen prüft deterministischer Code.
- 7. **Modellagnostische Provider–Abstraktion für LLM/CV/TTS.** Kein direkter, fest verdrahteter Aufruf eines einzelnen Anbieters in der Kernlogik — Provider ist austauschbar.
- 8. **Emotionale/Zustandssignale nur aus expliziter, granularer Nutzerzustimmung.** Keine implizite Verhaltens– oder Biometrie–Auswertung ohne gesonderten Consent.

Tech–Stack (Referenz)

- **Frontend:** Flutter (iOS, Android, Web) mit Theming/Feature–Flags pro Tenant
- **Backend:** FastAPI oder NestJS (Entscheidung projektabhängig, konsistent halten)
- **Datenbank:** PostgreSQL + PostGIS; Vektorindex pro Tenant (z. B. pgvector)
- **Event–Bus:** für Live–Kontext–Daten (Wetter, Verkehr, Events) — kein synchrones Ketten–Polling
- **Objektspeicher:** S3–kompatibel für Bilder/3D–Assets/Audioguides
- **Karten/Routing:** OpenStreetMap/GraphHopper als lizenzunabhängige Basis, Google Maps optional pro Tenant
- **Deployment:** containerisiert, cloud–agnostisch (Kubernetes), Region–Deployment möglich

Wie mit Aufgaben umgehen

- Vor größeren Änderungen: kurz prüfen, ob die Aufgabe eines der Prinzipien oben berührt — falls ja, das explizit im Vorschlag benennen, nicht stillschweigend umgehen.
- Neue Datenquellen/Anbieter: immer als Adapter mit klarem Interface anlegen, auch wenn aktuell nur ein Anbieter existiert.
- Bei Unsicherheit, ob etwas Kernlogik oder Tenant–Konfiguration ist: im Zweifel als Konfiguration behandeln — das ist der teurere Fehler, wenn er andersherum passiert.
- Tests für Tenant–Isolation (Daten aus Tenant A dürfen nie in Tenant B sichtbar sein) sind Pflichtbestandteil, nicht optional.

Phasenrahmen

Phase	Fokus
1 — MVP	Trip Planning + Content für 1 Pilotmarkt, Multi–Tenant–Grundgerüst steht
2	CV/AR–Guide, Live–Context–Engine
3	Booking–Agent, B2G–Dashboard
4	Weitere Länder/Tenants — reines Onboarding (Content + Config), keine Kernentwicklung

Lackmustest: Wenn Phase 4 eine Code–Änderung statt nur Konfiguration/Content braucht, war die Trennung in einer früheren Phase nicht sauber genug — das ist ein Signal, die Architektur zu überprüfen, nicht einfach weiterzumachen.